SmartScheduler

SmartScheduler is a multi-stage system designed to perform fair and constraint-aware hospital shift scheduling by combining the reasoning capabilities of Large Language Models (LLMs) with symbolic verification and Planning and Constraint (PC) techniques. The system aims to automatically generate balanced work schedules that satisfy institutional requirements while also considering the personal preferences and well-being of healthcare workers.

The framework is structured into four main stages:

1. **Preferences Definition,**
2. **Schedule Drafting,**
3. **Schedule Verification,**
4. **Schedule Refinement**.

Input

The agentic framework receives as input a text file containing a model draft containing information about:

- available shifts,
- number of available workers,
- hard constraints coding scheduling legal requirements.

Stage 1: Preferences Definition

In this stage workers' scheduling preferences are collected and formalized using natural language interaction.

Workflow

Preferences specification

Healthcare workers provide, using natural language, the following information:

- Preferred shifts (e.g., morning, afternoon, night)
- Availability constraints
- Tolerance toward undesirable shifts such as:
    - Night shifts
    - Holiday shifts
    - Consecutive demanding shifts

Examples:

- "Worker A prefer morning shifts and I would like to avoid night shifts whenever possible."
- "Worker B can work during weekends, but not on consecutive holidays."

- “Worker A is available for emergency coverage twice a month.”

Natural Language Understanding

An LLM-based *workers agent* interprets worker’s textual preferences and transforms them into scheduling hard constraints and soft preferences expressed as a python file containing the OR-tools corresponding specification.

Preference Formalization

The extracted information is converted into a machine-readable representation suitable for scheduling and optimization algorithms.

The system distinguishes between:

- **Hard constraints**
  - Legal requirements
  - Minimum staffing levels
  - Maximum working hours
  - Mandatory rest periods
- **Soft constraints**
  - Personal preferences
  - Shift desirability
  - Individual tolerance levels

Preference Scoring

Each worker is associated with a satisfaction model that quantifies how well a generated schedule satisfies their preferences.

Stage 2: Schedule Drafting

Generate an initial hospital shift schedule that satisfies all hard constraints while maximizing workers’ satisfaction.

Schedule Generation

An LLM based *drafting agent* generates an initial plan, consisting of a python file in which workers preferences and shifts assignment are coded using or-tools specification, by considering:

- Hospital staffing requirements
- Shift coverage constraints
- Workers’ availability
- Workers’ expressed preferences

The generated schedule includes:

- Worker-to-shift assignments
- Shift rotations
- Holiday and night shift distributions
- Rest periods

Fairness-Oriented Allocation

The system attempts to distribute undesirable shifts fairly among workers accordingly with the extracted preferences. The objective is not only to maximize total satisfaction but also to avoid penalizing specific workers disproportionately.

Stage 3: Schedule Verification

In this stage, the correctness and fairness of the generated schedule are evaluated. The objective is to evaluate schedule compliance with legal (strict) rules, and then, if compliance is confirmed, to assess schedule fairness.

Hard Constraint Verification

A symbolic *verification agent* checks whether the generated schedule satisfies all mandatory scheduling constraints, including:

- Staffing requirements
- Legal work limits
- Rest constraints
- Shift compatibility rules

If violations are detected, the schedule is rejected and sent back to the *drafting agent* for revision.

Fairness Evaluation

A symbolic *fairness verification agent* compute fairness metrics over the generated schedule. The verification process identifies the most disadvantaged worker compared to the others.

Stage 4: Schedule Refinement

This stage aims at iteratively improving the schedule fairness while preserving all hard constraints.

Workflow

Drafting Agent Callback

The *drafting agent* is asked to refine its previous plan to improve the satisfaction of the least satisfied worker, identified by the *verification agent*, by considering the preferences and associated priorities declared by all workers.

The refinement process may include:

- Reassigning undesirable shifts
- Balancing workloads
- Improving satisfaction of disadvantaged workers
- Reducing scheduling inequality

Fairness Optimization Criterion

The refinement process focuses on improving the condition of the least satisfied workers. Fairness improvement is achieved when improving the satisfaction of the least satisfied worker does not worsen the minimum satisfaction level among the other workers.

Iterative Improvement Loop

The refinement cycle continues until:

- No additional fairness improvement is possible
- All hard constraints remain satisfied

Project Files to be Delivered

The final project delivery will include:

- **Project files:** a zip file with the implemented code.
- **Example of a SmartScheduler output:** The input partial OR-tools sat cp_model, and the final resulting scheduling.
- **Short Relation** describing the approach used in the project and the design choices, and discussing the quality of the results in the proposed use cases.

Scenarios to tackle

The system will be evaluated on two use cases:

- **Use case A:** Scheduling manages homogeneous workers; thus, every shift can be covered by any of the workers. This scenario includes 13 workers. At least two workers must be assigned to each shift.
- **Use case B:** Workers can be either “standard” or “specialized”. In this scenario, at least two standard workers and one specialized worker must be assigned to each shift. If needed, a specialized worker can also play the role of a standard one (e.g., a shift may be covered by

one standard and two specialized workers).
This use case includes 13 standard workers and 7 specialized workers.

In both use cases, each employee's working time cannot exceed 36 hours per week. Each worker must cover 25 shifts in a month. Each day includes three shifts: morning (8-14), afternoon (14-20), and night (20-8). The workload associated with the first two corresponds to a single shift, while the last corresponds to a double shift due to its much longer duration. It is mandatory to ensure each employee two free days after each night shifts.

Additionally, each worker can cover up to one shift per day and cannot cover two subsequent shifts. Moreover, each worker must be ensured a day of rest, regarding which workers can express preferences.

The scheduling horizon is one month, specifically, from the 7th of December 2026 to the 6th of January 2027.